



Project Proposal (Synopsis) Of

BACHELOR OF COMPUTER APPLICATIONS

(BCA)

On

HOTEL ROOM BOOKING WEBSITE

WITH

ASSISTANT CHATBOT

To,
Project Co-Ordinator (BCA)

INDIRA GANDHI NATIONAL OPEN UNIVERSITY (SCHOOL OF COMPUTER AND
INFORMATION SCIENCE (SOCIS))

Submitted By: _____

Enrollment No :2350526613

Name :Nikhil P N

User Guidance of: _____

ANJALI V

EnrollmentNo:2350526613

Name: Nikhil p n

BCSP-064

Hotel Room Booking Website with Assistant Chatbot

_____this is photostat page-----No Need To Write
here..performa page

Guide Details

Anjali V

E-mail: anjaliv238@gmail.com

Mobile No: 91+9847886086

Vattaparambath House, Kaipilly, Ringroad, Arimpur P.O, Thrissur, Kerala

Career Objective

To work in a firm with a professional work driven environment where I can utilize and apply my knowledge and skills in software development. I aim to contribute my expertise in web technologies and continue to grow professionally while fostering the company's success.

Technical Skills

Languages / Technologies: Angular, Angular JS, PHP (Yii1, Yii2, Laravel), JavaScript, HTML, CSS, Ajax

Database: MySQL, PgSQL

Operating Systems: Windows

Experience

Currently working in Tata Consultancy Services (TCS)

Previous Company: RM Education Solutions India Pvt Ltd (17/02/2020 -08/05/2023)

Role: Software Engineer

Roles & Responsibilities:

- 5 years of experience in design and development of applications using Angular, Angular JS, PHP, SQL and related web technologies
- Extensive experience in Yii1 & Yii2 framework with MVC architecture
- Liaison with clients to build confidence and delivering projects in time
- Working on Assessment Master platform - a comprehensive assessment management system

- Development of Designer Module for creating and previewing items (questions) and testlets (test papers)
- Implementation of Admin Module for deploying testlets and assigning to different learners
- Development of Player Module where learners can write exams both online or offline using USB player
- Creation of Marker Module for marking completed exams with roles like marker and supervisor
- Implementation of Session Module for managing testlets, learners, and scheduling exams as sessions
- Development of Reports Module for generating learner response reports and itemized learner response reports
- Career oriented, hardworking and ability to handle multiple tasks
- Good in managing time and organizing tasks

Education

- **B.Tech** - Thejus Engineering College - 7.52 CGPA - 2019
- **Higher Secondary School Certificate (HSC)** - St. Aloysius HSS, Elthuruth, Thrissur - 89.4% - 2015
- **Secondary School Certificate (SSLC)** - St. Anne's C.G.H.S West Fort, Thrissur - 97% - 2013

Personal Skills

- Good analytical skills
- Positive approach towards problems
- Self motivated
- Good team player
- Quick learner and ability to adapt to new technologies
- Good in managing time and organizing tasks

Field of Interest

- Problem Solving
- Web Development
- Learning New Technologies

Projects

Project Title: Assessment Master Platform

Language Used: Angular, Angular JS, PHP (Yii1, Yii2)

Database: PgSQL, MySQL

Description: A comprehensive assessment management system that helps organizations worldwide to create, deliver, mark and certify engaging, authentic, and accessible assessments. The platform includes multiple modules - Designer, Admin, Player, Marker, Session, and Reports.

Project Title: Web Application Development

Language Used: Laravel, PHP

Database: MySQL

Description: Development of various web applications using Laravel framework with MVC architecture.

Personal Information

Gender: Female

Marital Status: Married

Date of Birth: 17/10/1997

Nationality: Indian

Hobbies

- Listening to Music
- Playing Badminton

Languages Known

- English
- Malayalam
- Tamil
- Hindi

EnrollmentNo:2350526613

Name: Nikhil p n

BCSP-064

Hotel Room Booking Website with Assistant Chatbot

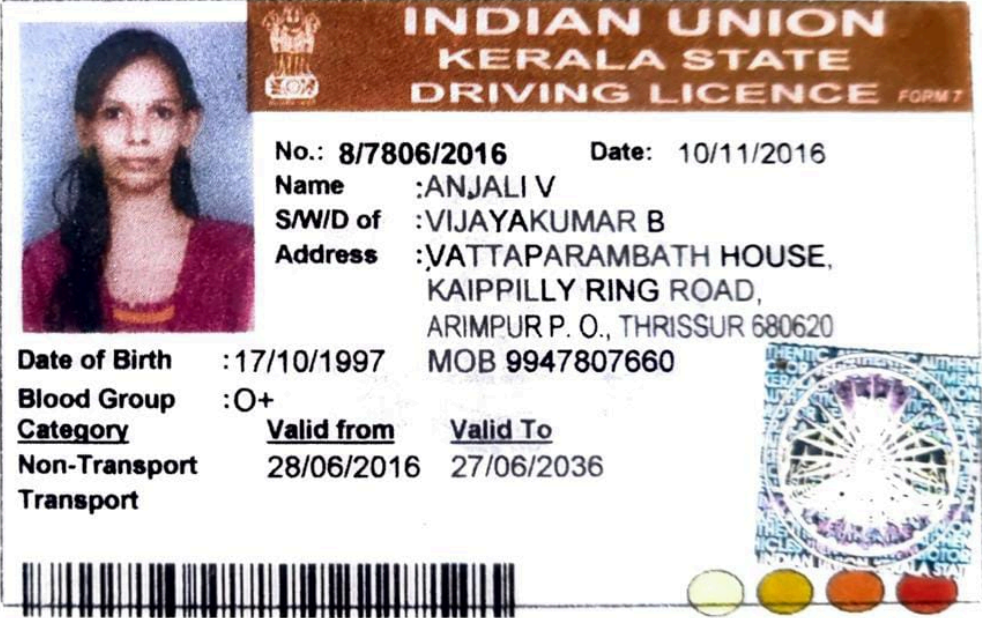
Contact Information of Guide

Permanent Address: Vattaparambath House, Kaipilly, Ringroad, Arimpur P.O, Thrissur, Kerala - 680620

Present Address: Vattaparambath House, Kaipilly, Ringroad, Arimpur P.O, Thrissur, Kerala - 680620

Cell Phone: 9847886086

E-Mail: anjaliv238@gmail.com



**INDIAN UNION
KERALA STATE
DRIVING LICENCE FORM 7**

No.: 8/7806/2016 **Date:** 10/11/2016

Name : ANJALI V

S/W/D of : VIJAYAKUMAR B

Address : VATTAPARAMBATH HOUSE,
KAIPPILLY RING ROAD,
ARIMPUR P. O., THRISSUR 680620
MOB 9947807660

Date of Birth : 17/10/1997

Blood Group : O+

Category **Valid from** **Valid To**

Non-Transport 28/06/2016 27/06/2036

Transport

Barcode: [Barcode]

Color calibration: [Yellow, Green, Blue, Red circles]

XI. CERTIFICATE OF ORIGINALITY

This is to certify that the project report entitled “**Hotel Room Booking Website with Assistant Chatbot**” submitted to **Indira Gandhi National Open University (IGNOU)** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Computer Applications (BCA)** is an **authentic and original work** carried out by **Mr. Nikhil P N**, bearing **Enrolment Number: 2350526613**, under my guidance.

The matter embodied in this project is **genuine work done by the student** and has **not been submitted** to this University or to any other University / Institute for the fulfilment of the requirements of any other course of study.

Signature of the Student:

Signature of the Guide

Name and Address of the student

Name, Designation and Address of Guide

Nikhil p n

Anjali v

**pokkaruparambil(H)
Thanissery (p o)
Irinjalakuda, Thrissur
Enrolment No: 2350526613**

**Anjali V Vattaparambath House Kaipilly,
Ringroad Arimpur P.O Thrissur,**

HOTEL ROOM
BOOKING WEBSITE
WITH
ASSISTANT
CHATBOT

TABLE OF CONTENT

1. Introduction of Project	10
2. Objectives of the Project	12
3. Project Category	13
4. Tools and System Requirements	15
5. Design of Project	
• DFD	16
• ERD	18
• Data Structure	19
• Database Design	24
• No of Modules and their Description	25
• Testing Details	28
6. Validation to be Performed	29
7. Types of Report Generation	30
8. Limitations of the Project	31
9. Future Scope of the Project	32
10. Bibliography	33

Introduction

About The Documentation:

The project documentation provides detailed information about the **existing system**, whether it is **manual or automated**, and explains how the current system performs the required functionalities. The documentation clearly exposes the **limitations and problems** of the existing system that are not currently addressed but need to be resolved in the future through the **proposed system**.

All identified problems are **clearly listed and well explained** so that they are easily understandable by any reader. The documentation includes the **complete scenario of both the existing and proposed systems**, along with the **objectives of the project, existing system requirements, and proposed system objectives**.

The documentation also reflects the **basic requirements for implementing the proposed software**, including an overview of the **estimated software expenses**. In addition, it specifies the **software development model** that will be followed during the development of the system.

During the documentation phase of this project, all the essential aspects of **good software documentation** have been carefully considered, such as **feasibility study, options study, and comparative study**. Furthermore, all topics mentioned in the **IGNOU Project Proposal guidelines**—including **Introduction, Objectives, System Analysis, Coding, Process Logic, and Implementation details**—have been properly addressed.

Proposed system

The proposed system allows customers to browse room types with images and descriptions, check real-time availability, and book rooms online with secure payment through a payment gateway such as Razorpay. It stores all booking, customer, and payment details in a relational database using Django ORM, preventing double-booking through date-range overlap checks and generating digital confirmations via email and downloadable PDFs for each successful reservation.

Administrators can manage room types, rooms, staff details, and customer contact messages through a dedicated backend dashboard, as well as view complete booking histories and total revenue records. The system further enhances usability and support by including user registration/login, password reset via email OTP, instant JavaScript notifications, and a planned intelligent chatbot module that will answer queries about room availability, pricing, and staff information directly from stored system data

Objectives of the project

- Provide a seamless online room booking experience with real-time availability checks and prevention of double booking using date-range overlap validation.
- Simplify hotel operations by allowing admins to create, update, and delete room types, rooms, and staff details directly from the backend panel.
- Enable secure user registration, login, and password reset via email OTP, ensuring authenticated access and account recovery.
- Facilitate two-way communication through a customer contact form and admin view of all contact messages.
- Integrate payment processing using Razorpay and store the final amount paid by each customer.
- Generate digital records such as booking details and reports in PDF format for download and future reference.
- Enhance user experience through JavaScript-based instant notifications and a chatbot that can answer queries about rooms, staff, and bookings.
- Provide a flexible room search feature that allows users to quickly find rooms by room name, as well as filter by room type, price, and availability dates, so they can directly locate specific rooms they are interested in.

PROJECT CATEGORY

Project Category: RDBMS-based Web Application (Client–Server, Three-Tier Architecture) using Django and PostgreSQL. This project follows a Client–Server architecture using a three-tier, web-based design. The client (browser) and server (Django application with database) communicate over a network, clearly separating user interface, business logic, and data storage.

In Client–Server Architecture, data management and transaction processing are handled on the server, independent of the client-side interfaces. The client (front-end web pages) is responsible for capturing user input and displaying responses, while the server executes business rules, processes room bookings, checks availability, and interacts with the database, which improves maintainability, security, and performance.

The front-end (client) is built with HTML, CSS, Bootstrap, and JavaScript and runs in the user's browser, providing forms for signup, login, room search, booking, contacting the hotel, viewing staff, making payments, chatting with the assistant, and downloading PDFs. The back-end (server) is implemented using Django and PostgreSQL, where the server processes HTTP requests, validates input, checks for overlapping bookings, manages sessions, generates OTPs, sends emails, interacts with Razorpay for payments, and reads/writes all records in the relational database.

Data travels between client and server over HTTP/HTTPS in a structured format (such as URL-encoded form data or JSON responses) and is mapped by Django views and models into tabular records inside the PostgreSQL database. The major advantage of this Client/Server approach is that the same server can handle requests from many users at the same time, with the browser handling presentation and basic interaction while the server centralizes core logic and data consistency.

This project also adopts a three-tier architecture:

- User Services (Presentation Tier) – Browser-based UI pages for customers and admins (HTML, CSS, Bootstrap, JavaScript).
- Business Services (Application Tier) – Django views, URL routing, and business logic for booking, payments, OTP verification, email notifications, PDF generation, chatbot integration, etc.
- Data Services (Data Tier) – Django models and PostgreSQL tables (`roomtypedb`, `roomnamedb`, `Registerdb`, `bookingdb`, `Totaldb`, `customercontactdb`, `staffdb`) that handle persistent storage, validation, and relationships.

Using a three-tier Client–Server architecture with PostgreSQL allows concurrent multi-user access, easier scaling (frontend, backend, and database can be deployed separately), and smoother future enhancements such as adding more services or upgrading the chatbot without major changes to other layers.

TOOLS / PLATFORM, HARDWARE AND SOFTWARE **REQUIREMENT SPECIFICATION**

Front End

- HTML5, CSS3, Bootstrap – for responsive user interface design.
- JavaScript – for client-side validation and instant notifications (alerts/toasts).

Back End

- Python
- Django Web Framework (MTV architecture) – for request handling, business logic, authentication, email, and ORM.

Database

- PostgreSQL – relational database for storing users, rooms, bookings, payments, staff, and contact messages.

Web / Application Server

- Django development server for local use, deployable on Apache / Nginx with Gunicorn or similar WSGI server.

External Services / Libraries

- Razorpay Payment Gateway – for online payment processing.
- xhtml2pdf – for generating downloadable PDF booking reports.
- Django Email utilities / SMTP (Gmail) – for OTP and booking confirmation emails.

Development / Documentation Tools

- Code Editor/IDE: VS Code / PyCharm.
- Version Control: Git and GitHub for source code management.
- Documentation & Reports: Google Docs / Microsoft Office (Word, Excel, PowerPoint) for synopsis, SRS, and project report.

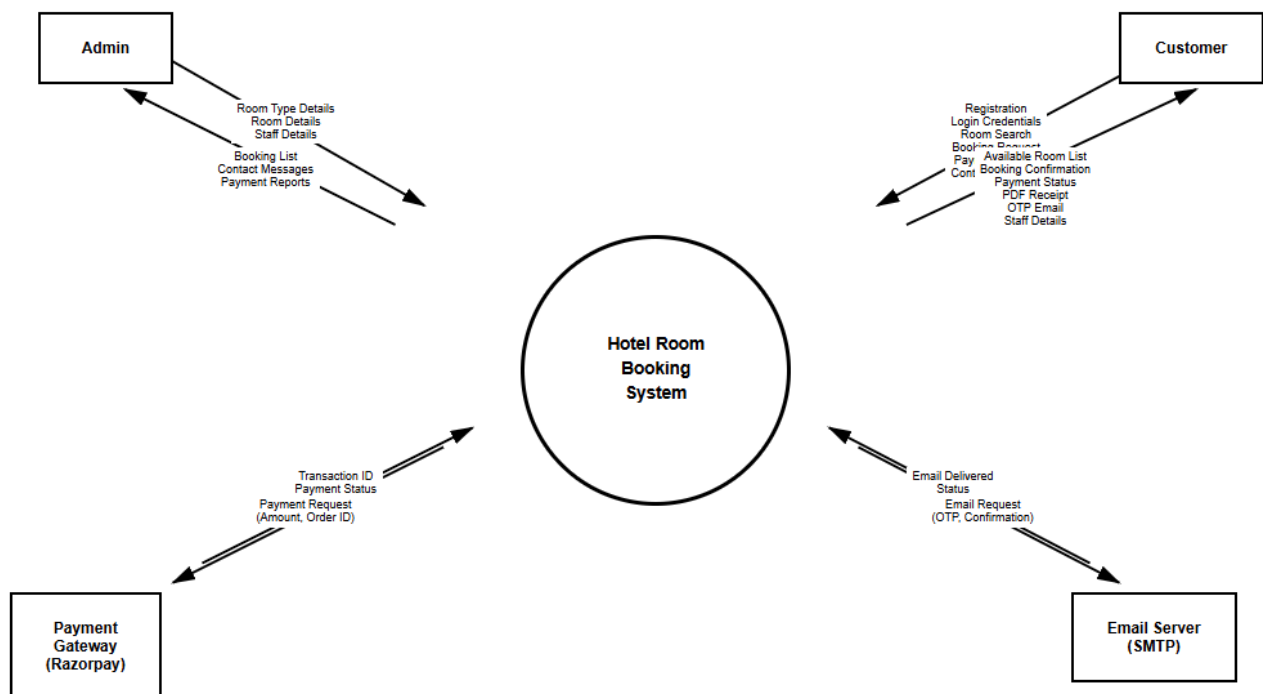
Hardware Requirements (Minimum)

- Processor: Dual-core or higher.
- RAM: 4 GB (8 GB recommended for smooth development).
- Storage: 50 GB free disk space for OS, tools, database, and project files.
- Network: Internet connection for email, payment gateway, and GitHub access.

DATA FLOW DIAGRAM

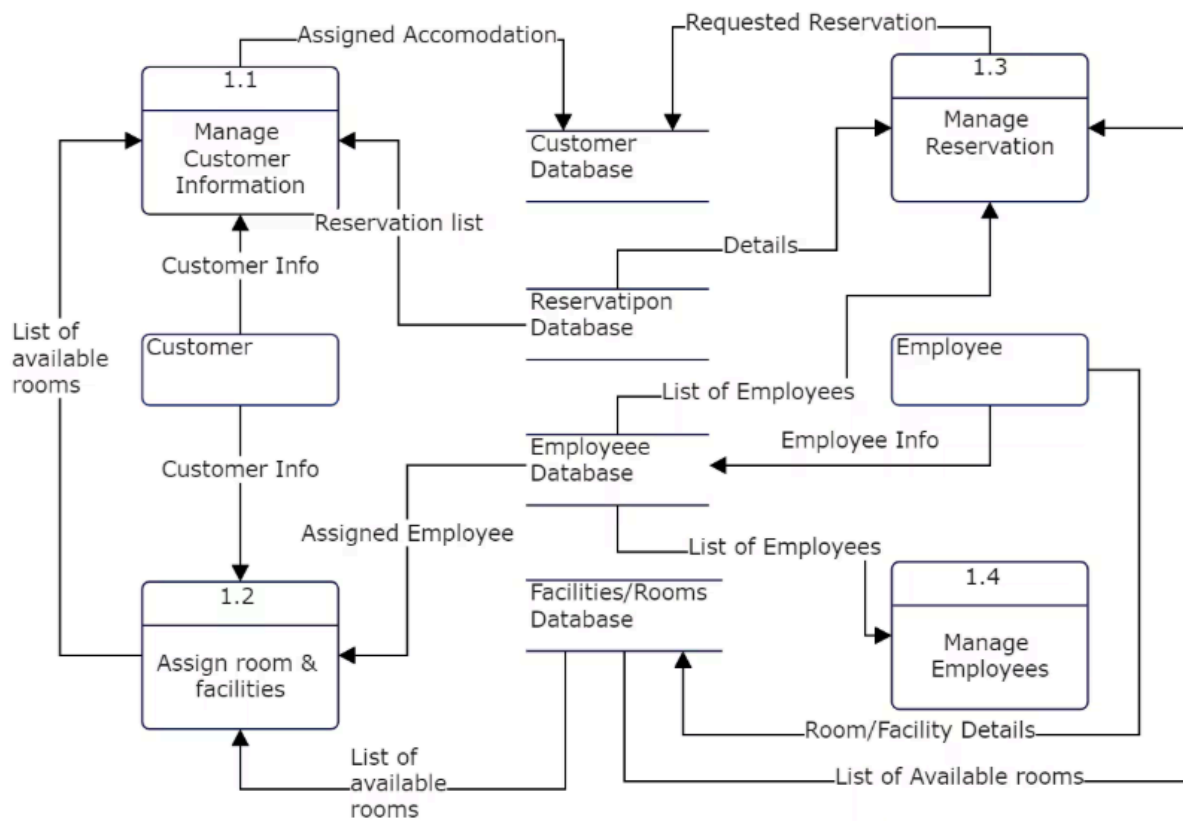
Hotel Room Booking System

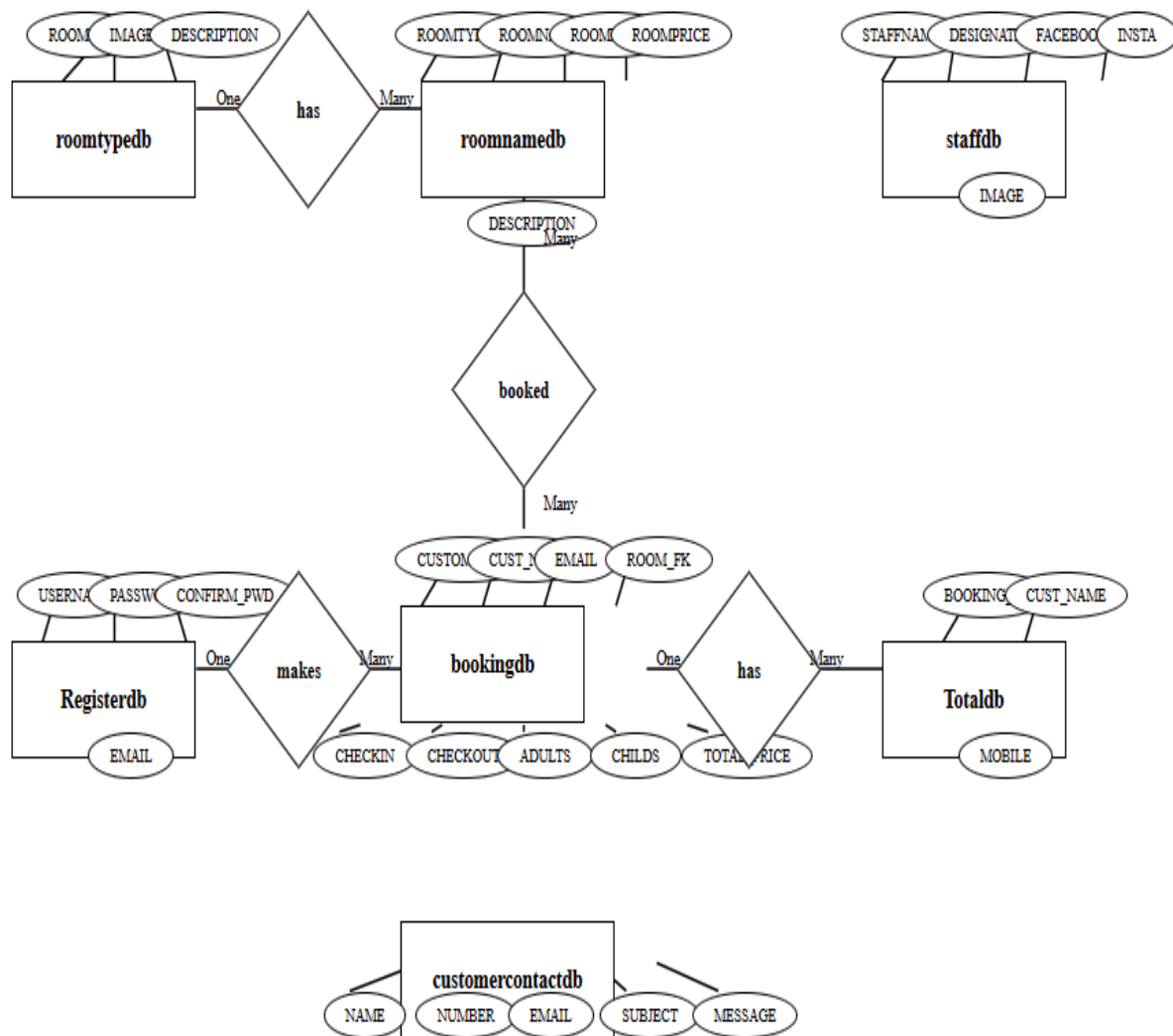
DIAGRAM 1: CONTEXT LEVEL DFD



DFD FOR HOTEL ROOM BOOKING SYSTEM (LEVEL 1)

DFD for Hotel Management System



ENTITY-RELATIONSHIP DIAGRAM

DATA STRUCTURE

TABLE: ROOMTYPE_MASTER (roomtypedb)

Field name	Data Type	Constraints	Description
RoomType_ID	INT	PK, Not null, Auto increment	Unique ID of room type
ROOMTYPE	VARCHAR(100)	Not null	Name of room type (e.g., Deluxe)
ROOMTYPEIMAGE	VARCHAR(255)	NULL	Path of room type image
DESCRIPTION	VARCHAR(500)	NULL	Description of room type

TABLE: ROOM_MASTER (roomnamedb)

Field name	Data Type	Constraints	Description
Room_ID	INT	PK, Not null, Auto increment	Unique ID of room
RoomType_ID	INT	FK ROOMTYPE_MASTER(RoomType_ID), Not null →	Room type reference
ROOMNAME	VARCHAR(100)	Not null	Room name / number
ROOMIMAGE	VARCHAR(255)	NULL	room image

ROOMPRICE	INT	Not null	Price
ROOMDESCRIPTION	VARCHAR(500)	NULL	Description of room

TABLE: USER_MASTER (Registerdb)

Field name	Data Type	Constraints	Description
Customer_ID	INT	PK, Not null, Auto increment	User identity
USERNAME	VARCHAR(100)	Not null	User name
PASSWORD	VARCHAR(100)	Not null	Login password
CONFIRMPASSWORD	VARCHAR(100)	Not null	Confirm password
EMAIL	VARCHAR(100)	Not null	email address

TABLE: BOOKING_MASTER (bookingdb)

Field name	Data Type	Constraints	Description
Booking_ID	INT	PK, Not null, Auto increment	Booking identity
Customer_ID	INT	FK → USER_MASTER(Customer_ID), NULL allowed	Registered user reference
CUSTOMERNAME	VARCHAR(100)	Not null	Customer name (for display)
CONTACTEMAIL	VARCHAR(100)	Not null	Customer email
Room_ID	INT	FK → ROOM_MASTER(Room_ID), Not null	Selected room
CHECKIN	DATE	Not null	Check-in date
CHECKOUT	DATE	Not null	Check-out date
TOTALADULTS	INT	Not null	Number of adults
TOTALCHILDS	INT	NULL	Number of children
SPECIALREQUEST	TEXT	NULL	Special requests of customer

TOTALPRICE	INT	Not null	Total booking price
------------	-----	----------	---------------------

TABLE: PAYMENT_TOTAL (Totaldb)

Field name	Data Type	Constraints	Description
Total_ID	INT	PK, Not null, Auto increment	Payment identity
Booking_ID	INT	FK BOOKING_MASTER(Booking_ID), NULL allowed →	Related booking
CUSTOMERNAME	VARCHAR(100)	Not null	Customer name
MOBILE	VARCHAR(15)	Not null	Customer mobile number
TOTALPRICE	INT	Not null	Total amount paid

TABLE: CUSTOMER_CONTACT (customercontactdb)

Field name	Data Type	Constraints	Description
Contact_ID	INT	PK, Not null, Auto increment	Contact identity
CONTACTNAME	VARCHAR(100)	Not null	Customer name

CONTACTNUMBER	VARCHAR(15)	Not null	Customer phone number
CONTACTEMAIL	VARCHAR(100)	Not null	Customer email
CONTACTSUBJECT	VARCHAR(200)	Not null	Subject of message
CONTACTMESSAGE	TEXT	Not null	Message body

TABLE: STAFF_MASTER (staffdb)

Field name	Data Type	Constraints	Description
Staff_ID	INT	PK, Not null, Auto increment	Staff identity
STAFFNAME	VARCHAR(100)	Not null	Staff name
STAFFDESIGNATION	VARCHAR(100)	Not null	Designation/role
STAFFFACEBOOK	VARCHAR(100)	NULL	Facebook link
STAFFINSTA	VARCHAR(100)	NULL	Instagram link
STAFFIMAGE	VARCHAR(255)	NULL	Path of staff image

Database Design

The system uses a relational database in PostgreSQL, designed with normalized tables and proper primary–foreign key relationships to avoid redundancy, anomalies, and to maintain data integrity throughout all transactions. Each logical entity of the application—room type, room, user, booking, payment, staff, and customer contact—is represented as a separate table, with carefully chosen data types and constraints so that updates, inserts, and deletes are consistent and safe.

The core tables are roomtypedb (room category), roomnamedb (individual rooms), Registerdb (users), bookingdb (bookings), Totaldb (payments), customercontactdb (contact messages), and staffdb (staff details). These tables are interlinked using foreign keys so that each booking is associated with one registered user and one room, and each payment record is tied to a specific booking, which makes it easy to track complete booking and payment history for every customer. Referential integrity constraints ensure that a room type cannot be deleted while rooms of that type exist, a room cannot be removed while it has bookings, and a booking cannot be removed while its payment record is present, thus preventing orphan records.

The design supports efficient querying through indexing of frequently searched fields such as room type, room id, booking dates, and customer id, which improves performance for operations like room search, availability checking, and report generation. Date fields for check-in and check-out, integer fields for number of guests and prices, and text fields for special requests and contact messages together provide a rich but structured schema that can handle all current requirements while remaining flexible for future enhancements such as additional tables for reviews, coupons, or multi-hotel support.

NUMBER OF MODULES AND THEIR DESCRIPTION

The following are the major activities of the Hotel Room Booking System. It contains a number of modules with their brief description. The modules used in this software are:-

1. Secure Login & User Management
2. Room Type & Room Management
3. Staff Management
4. Customer Contact Management
5. Room Search & Booking Management
6. Payment & Total Amount Module
7. Notification, Email & PDF Report Module
8. Assistant Chatbot Module

1. Secure Login & User Management

Secure Login:

This is the first activity of the system and allows only authorized users to access customer features or the admin backend. It verifies username and password against the registered users table and stores the username in session so that unauthorized people cannot view, modify, or delete data inside the application.

User Management:

This part handles the records of users who will use the system. It allows new users to sign up, existing users to sign in, sign out, and reset forgotten passwords using an OTP sent to their registered email, ensuring proper account recovery and controlled access.

2. Room Type & Room Management

Whenever a new room type (for example Deluxe, Suite, Standard) or a new room is added in the hotel, this module is used. It enables the admin to insert, update, and delete room types and individual rooms, including images, descriptions, and prices, so that the online catalogue always matches the actual inventory of the hotel.

3. Staff Management

When new staff members join or existing staff details change, the Staff Management module is used. It allows the admin to maintain staff name, designation, social-media links, and photos, and these details are displayed on the About/Our Team pages so that customers can know about the team behind the hotel services.

4. Customer Contact Management

When a visitor sends a message through the contact form, he or she is treated as a customer within the system. This module stores and manages details such as customer name, email, mobile number, subject, and message so that the admin can read, respond, and delete messages from a single backend page, improving hotel–guest communication.

5. Room Search & Booking Management

This is one of the main functional modules of the software. It allows a logged-in customer to search rooms by room type, room name, date range, and number of guests and then place a booking request. The module checks availability by validating date-range overlaps for the selected room, prevents double booking, stores booking details such as check-in, check-out, adults, children, special requests, and calculates the total price.

6. Payment & Total Amount Module

After confirming the booking, this module calculates subtotal, tax, and final total amount to be paid by the customer. It integrates with the Razorpay payment gateway to create an order, process online payment, and then stores the final paid amount, customer name, and mobile in a separate table for future reference and financial reporting.

7. Notification, Email & PDF Report Module

This module handles user-friendly communication and reporting features. It sends booking confirmation emails, password reset OTP emails, and password-change alerts to customers, generates PDF documents for booking details using a report template, and uses JavaScript-based instant notifications to inform users about successful operations like booking, message submission, or data updates.

8. Assistant Chatbot Module

The assistant chatbot provides an interactive way for customers to communicate with the hotel website. It can answer questions about available rooms, prices, facilities, staff information, booking status, and guide the user to the correct page or action, thereby reducing manual support work for the staff and improving customer satisfaction.

Testing Details

The system will be tested using unit testing, integration testing, and system testing to ensure that all modules work correctly and together as a whole. Unit tests will verify individual functions such as user registration, login validation, room availability check, booking creation, payment calculation, and OTP verification, while integration tests will check the data flow between modules like booking–payment–email and backend–database interactions.

User interface testing will confirm that all forms accept valid data, reject invalid input, and display proper messages, and functional test cases will cover common scenarios such as successful booking, double-booking rejection, payment success/failure, password reset, and contact form submission. Final acceptance testing will be done with sample users to validate that the system meets the project objectives and performs reliably under normal load

VALIDATIONS TO BE PERFORMED

- Every registered user should have a unique Customer_ID that is generated automatically when saving a new record in the USER_MASTER (Registerdb) table.
- All compulsory fields (* marked fields) such as username, email, password, room selection, check-in date, check-out date, and total price must not be left blank in the signup and booking forms.
- In password fields, characters typed by the user must be masked with symbols (such as ● or *) so that the actual password is not visible on the screen.
- The system must not accept alphabetic characters or negative values where only numeric amounts are required, such as room price, number of adults, number of children, and total amount to be paid.
- Proper date format validation must be applied for CHECKIN and CHECKOUT at the time of data entry, and the user should be allowed to correct the dates if they are invalid or in the wrong order (CHECKOUT must be after CHECKIN).
- A booking should not be saved if the selected room is already booked for any overlapping date range, ensuring no double booking for the same room and period.
- Master information such as room types, rooms, and staff records that are already used in existing bookings should not be deleted directly, or the system should warn the admin and prevent deletion if active transactions exist.
- Any unauthorized person cannot enter into the system because each user is validated using username and password at login; only authenticated users can access booking, payment, and history pages, while admin pages are further restricted to admin credentials.

TYPES OF REPORT GENERATION

- **Customer Booking Report**
Displays the details of customers who have booked rooms, including customer name, email, mobile, selected room, check-in and check-out dates, number of guests, and total price.
- **Room Booking Summary Report**
Displays a summarized list of all bookings for each room, showing room type, room name, booking dates, occupancy, and current booking status, helping the admin analyze room usage and availability.
- **Payment / Revenue Report**
Displays the payment details stored in the Totaldb table, including customer name, mobile number, booking reference, and total amount paid, useful for daily, weekly, or monthly revenue tracking.
- **Customer Contact Report**
Displays all messages submitted through the contact form, listing customer name, email, phone, subject, and message so that the hotel can follow up on enquiries and feedback.
- **Staff Details Report**
Displays staff information such as name, designation, and contact/social links, which can be used for internal records or to generate the “Our Team” information shown on the website.

LIMITATIONS OF THE PROJECT

- Needs a modern web browser and an operating system capable of running a Python/Django environment (for example Windows 10 or higher, recent Linux distribution, or equivalent).
- Requires Python, Django, PostgreSQL, and supporting libraries (Razorpay SDK, xhtml2pdf, email/SMTP configuration) to be installed and properly configured on the server machine.
- For smooth performance with multiple users, the development/deployment system should have at least a dual-core processor and 4–8 GB RAM; very low-end hardware may cause slower response times when handling bookings, payments, and PDF generation

FUTURE SCOPE OF THE PROJECT

This project is designed and developed in such a manner that it provides maximum efficiency and speed, and it has a wide scope for further development. A number of new modules can be added in future (such as dynamic pricing, loyalty programs, or multi-hotel support) with very few changes in the existing database structure and only minimal changes in the application code.

The system can be extended with more powerful query and reporting capabilities to generate advanced analytical reports on occupancy, revenue, customer behaviour, and staff performance. In future, deeper integrations such as AI-based chatbot services, external hotel booking channels, or advanced payment and refund workflows may also be incorporated, which are not part of the current implementation.

BIBLIOGRAPHY

Websites –

1. <https://docs.djangoproject.com/>
2. <https://www.python.org/doc/>
3. <https://www.postgresql.org/docs/>
4. <https://razorpay.com/docs/payments/server-integration/django/>
5. <https://getbootstrap.com/docs/>
6. <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
7. <https://xhtml2pdf.readthedocs.io/>

Reference books (Python / Django / Web development) –

1. Adrian Holovaty, Jacob Kaplan-Moss, *The Definitive Guide to Django*.
2. William S. Vincent, *Django for Beginners*.
3. Eric Matthes, *Python Crash Course*.
4. Robin Nixon, *Learning PHP, MySQL & JavaScript* (for general web concepts).

Reference books (Databases / SQL / PostgreSQL) –

1. Regina O. Obe, Leo S. Hsu, *PostgreSQL: Up and Running*.
2. Allen G. Taylor, *SQL For Dummies*.
3. Abraham Silberschatz, Henry F. Korth, S. Sudarshan, *Database System Concepts*.